Architecture Specification

Student: Shashank Arya

SRN: PES1UG24CS432

Problem Statement: #04 – Student Project Portfolio & Showcase App

Basis: Requirements explicitly supplied in the Lab 3 request. The Lab 3 handout and the Scrum, Kanban, and Jira Bug Tracker source documents were not present in the workspace at preparation time; document-specific details and defect IDs therefore remain unverified.

1. System Overview

The system lets student teams publish showcase profiles and multimedia artifacts, lets judges assess projects against digital rubrics, and lets verified public voters cast one vote per project. A live leaderboard reflects accepted votes. This is a logical component model, not a claim that an implementation or a particular deployment technology already exists.

2. Architectural Style

A layered, service-oriented component architecture separates the presentation boundary, project and voting business capabilities, and persistence/media services. The voting-to-leaderboard update path is event-driven: accepted vote events feed leaderboard aggregation, which publishes refreshed rankings to the presentation component. Synchronous interfaces handle profile, submission, evaluation, verification, and vote requests. The model does not prescribe a cloud provider, identity vendor, database, or transport protocol.

3. Components and Responsibilities

Component	Layer	Responsibility
Showcase Portal	Presentation	Presents team profiles, project details and artifacts; provides judge rubric entry and public voting/ranking views; routes user actions to business services.
Identity Verification	Application / Business	Verifies a voter's identity and supplies a verified voter reference for vote authorization and deduplication. The verification authority and assurance level must be selected from the actual project requirements.
Showcase Profile	Application / Business	Creates and serves team showcase profiles and the project catalog used by submissions, evaluation, voting, and ranking.
Project Submission	Application / Business	Accepts multimedia artifact metadata and uploads, associates artifacts with projects, and coordinates artifact storage and project records.
Evaluation & Rubric	Application / Business	Accepts judge rubric scores and comments and records evaluation results for a project.
Public Voting	Application / Business	Validates the target project and verified voter; atomically records at most one accepted vote per verified voter and project; emits an accepted-vote event.
Leaderboard & Realtime	Application / Business	Aggregates accepted votes by project, serves rankings, and publishes ranking updates after vote events.
Project Data Repository	Data	Persists showcase profile, project, submission metadata, and evaluation records through the project-data interface.
Vote Ledger	Data	Persists vote records and enforces a unique `(verifiedVoterRef, projectId)` key as the storage-level duplicate-vote guard; exposes vote counts/events to authorized services.

Component	Layer	Responsibility
Multimedia Artifact Store	Data	Stores uploaded media objects and returns stable artifact references for project metadata.

4. Interfaces and Contracts

A **provided** interface is a service offered by its provider component (UML lollipop). A **required** interface is a service consumed by its requiring component (UML socket). An assembly connector joins the matching ball and socket; arrows on data-flow annotations indicate the principal request or event direction, not UML interface ownership.

Interface	Provider	Requirer(s)	Main operations / exchanged data	Purpose
`IIdentityVerification`	Identity Verification	Public Voting	`verifyVoter(credential) -> verifiedVoterRef, status`	Establish a verified voter identity before accepting a vote.
`IShowcaseCatalog`	Showcase Profile	Showcase Portal; Project Submission; Evaluation & Rubric; Public Voting; Leaderboard & Realtime	`createProfile(team, project)`, `getProject(projectId)`, `listPublishedProjects()`	Create and retrieve the canonical team/project showcase record and publication state.
`IProjectSubmission`	Project Submission	Showcase Portal	`submitArtifact(projectId, file, metadata) -> artifactRef, status`	Submit project media and receive its stored reference/status.
`IArtifactStore`	Multimedia Artifact Store	Project Submission	`putObject(file, metadata) -> artifactRef`; `getObject(artifactRef)`	Persist/retrieve multimedia payloads independently of project metadata.
`IProjectData`	Project Data Repository	Showcase Profile; Project Submission; Evaluation & Rubric	`saveProfile(...)`, `saveProject(...)`, `saveArtifactReference(...)`, `saveEvaluation(...)`, `getProject(...)`	Persist/retrieve project, artifact-reference, and evaluation records.
`IRubricEvaluation`	Evaluation & Rubric	Showcase Portal	`submitEvaluation(projectId, judgeRef, rubricScores, comments) -> evaluationResult`	Record a judge's digital-rubric assessment.
`IPublicVoting`	Public Voting	Showcase Portal	`castVote(projectId, verifiedVoterRef) -> receipt or alreadyVoted`	Accept one vote per verified voter/project and report the outcome.
`IVoteLedger`	Vote Ledger	Public Voting; Leaderboard & Realtime	`recordVoteOnce(voterRef, projectId)`, `countVotes(projectId)`, accepted-vote records	Atomically enforce vote uniqueness and provide authoritative vote data.
`IAcceptedVoteEvents`	Public Voting	Leaderboard & Realtime	`VoteAccepted(projectId, voteRef, acceptedAt)` event stream	Notify the ranking service only after a vote has been accepted and committed.
`ILeaderboard`	Leaderboard & Realtime	Showcase Portal	`getRanking() -> ordered project scores`; `subscribeRankingUpdates()`	Serve current rankings and real-time ranking changes.

The vote ledger's unique key is a design control for FR-003. Identity verification raises the barrier to sybil voting (NFR-002), but the supplied requirements do not define an identity provider, eligibility policy, or assurance level; the architecture does not claim that verification alone eliminates all sybil attempts.

5. Component Relationships and Data Flow

1. A team uses the Showcase Portal to create a profile. Showcase Profile validates/catalogs the project and persists its records through IProjectData.

2. The team uploads an artifact through Project Submission. Submission associates metadata with the project, stores media bytes through IArtifactStore, and persists artifact references through IProjectData.

3. A judge submits rubric scores and comments through the portal. Evaluation & Rubric checks the project catalog and saves the evaluation through IProjectData.

4. A voter requests a vote through the portal, which forwards the verification material with the request. Public Voting checks the project catalog and calls Identity Verification; it then asks Vote Ledger to atomically record (verifiedVoterRef, projectId). A duplicate key returns an already-voted outcome rather than a second accepted vote. Credentials are not treated as verified merely because the portal supplied them.

5. After the vote transaction commits, Public Voting emits VoteAccepted. Leaderboard & Realtime consumes accepted-vote events, refreshes project totals/rank order from authoritative ledger data, and publishes ranking updates to the portal. The portal can also request a current ranking snapshot.

6. Requirement Coverage

Functional Requirements

Requirement	Supporting components and behavior
FR-001 Create Showcase Profile	Showcase Portal invokes Showcase Profile; project/profile records are persisted by Project Data Repository.
FR-002 Judge Project Using Digital Rubric	Showcase Portal invokes Evaluation & Rubric with project, judge, scores, and comments; results are persisted in Project Data Repository.
FR-003 Cast One Vote Per Project	Public Voting requires verified identity and project validity; Vote Ledger atomically enforces one `(verifiedVoterRef, projectId)` record.
FR-004 Upload Multimedia Artifacts	Showcase Portal invokes Project Submission; Multimedia Artifact Store holds media and project data holds artifact references/metadata.
FR-005 Display Project Leaderboard	Leaderboard & Realtime aggregates accepted votes and serves an ordered ranking through `ILeaderboard`.

Non-Functional Requirements

Requirement	Architectural support and limitation
NFR-001 Real-Time Voting and Leaderboard	Committed-vote events drive ranking refreshes and subscription updates. No latency or throughput target was supplied, so none is asserted.
NFR-002 Voter Identity Verification	Public Voting cannot accept a vote without a verified voter reference. The actual verification source, enrollment rules, privacy requirements, and sybil threat model require confirmation from the missing source documents.

7. Jira Defect Traceability

No Jira Bug Tracker document, defect export, issue key, or screenshot was available in the workspace. Therefore no project-specific defect is named, claimed reproduced, or claimed fixed here. The architecture includes a duplicate-vote storage constraint and verified-identity gate because these directly support the stated FR-003/NFR-002; these are requirement-derived controls, not assertions about Jira evidence. Add the actual defect IDs, descriptions, and verified component mappings after supplying the Jira report.

8. Design Decisions

• Separate profile/project metadata, vote records, and media payloads according to their different access and integrity needs.

• Enforce vote uniqueness in the authoritative ledger, not only in a portal/session check, so concurrent or repeated submissions share one constraint.

• Pass a verified voter reference into voting; do not couple the voting component to an unspecified identity vendor.

• Publish leaderboard events only after vote acceptance/commit, so rejected duplicate votes do not inflate rankings.

• Keep interfaces technology-neutral and components logical; deployment topology and operational guarantees need the actual SRS/handout before they can be specified.

• Treat the provided FR/NFR list as the available traceability baseline. Review against the original Scrum, Kanban, Jira, and Lab 3 documents before submission.

9. Source and Evidence Limitations

The workspace inventory was empty before deliverable creation. The Lab 3 handout, Scrum/Kanban exports, Jira Bug Tracker, screenshots, GitHub evidence, and implementation/test evidence were not available. The root README.md and 1-RE/ explain the exact manual additions needed. No evidence or implementation result is fabricated in this submission.